DOCKER

Storage & Memory Masterclass

BY AKSHAT GUPTA – Akshatawsac@gmai.com

Docker Prune | Volume Types | Named vs Unnamed | Bind Mounts | Memory Limits

Copyright @AkshatGupta

🧹 Docker Prune & Dangling Images

😱 THE PROBLEM

Netflix runs thousands of CI/CD pipelines daily. Each build creates a new Docker image. Old images pile up like stacked pizza boxes — **1 TB of disk gone in 48 hours.** Builds start failing. Engineers wake up at 3 AM.

✅ THE FIX

docker system prune -a
Removes all stopped containers, unused networks, dangling images, and build cache. Run it on a cron job every night.

💡 What Is a Dangling Image?

🏷️ A dangling image has no tag and no container using it — it's an orphan.

🔄 Created when you rebuild an image: the old version loses its tag but still sits on disk.

🔍 Spot them: docker images -f "dangling=true"

🗑️ Clean them: docker image prune (only dangling)

DOCKER VOLUMES

THE PROBLEM

A fast-growing company like Netflix runs thousands of CI/CD pipelines daily. Each pipeline builds and deploys containerized applications using Docker.

Everything seems fine… until one day:

- Users report missing data
- Applications restart → all stored data disappears
- Engineers realize logs, uploads, and DB data are not persistent
- New deployments = fresh containers with zero previous data

Docker Volume Types

Three ways to store data — each solves a different problem

Named Volume

WHEN TO USE

Persistent app data

```
docker run -v mydata:/app/data
```

KEY TRAITS

- ✓ Managed by Docker
- ✓ Easy to back up
- ✓ Survives container death

Best for: Databases, uploads, logs

Anonymous Volume

WHEN TO USE

Temporary data

```
docker run -v /app/tmp
```

KEY TRAITS

- ✓ Auto-created by Docker
- ✓ No name to remember
- ✓ Tied to container

Best for: Short-lived scratch space

Bind Mount

WHEN TO USE

Dev & config files

```
docker run -v /host/path:/app
```

KEY TRAITS

- ✓ Use existing host files
- ✓ Live code reload
- ✓ Full host access

Best for: Local development

🏷️ Named vs Unnamed Volumes

The difference that saves your data

😱 **NETFLIX's DATA LOSS INCIDENT**

A junior dev ran `docker run -v /app/data` without a name. Container restarted → volume orphaned. User uploads? Gone. 200GB of data lost overnight.

👻 Unnamed / Anonymous Volume

```
docker run -v /app/data myapp
```

- Docker picks a random UUID as name
- No easy way to reference it later
- Hard to back up or migrate
- Deleted when you run `docker volume prune`
- Default when you forget to name it

❌ **Risky for production**

🏷️ Named Volume

```
docker run -v userdata:/app/data myapp
```

- You give it a meaningful name
- Survives container stop/remove/restart
- Easily referenced in backups & scripts
- Only deleted with `docker volume rm yourname`
- Safe & predictable

✅ **Recommended for production**

Docker Volumes — Named Volume Deep Dive

The right way to persist data in production

How It Works

1. **Create**

```
docker volume create mydata
```

2. **Attach to container**

```
docker run -v mydata:/app/data
```

3. **Docker stores data**

```
/var/lib/docker/volumes/mydata/_data
```

4. **Container dies — data LIVES**

```
New container, same volume = data restored
```

Volume Command Cheatsheet

Create	`docker volume create mydata`
List all	`docker volume ls`
Inspect	`docker volume inspect mydata`
Delete	`docker volume rm mydata`
Prune orphans	`docker volume prune`
Use in run	`docker run -v mydata:/app/data img`
Use in compose	`volumes:` `  mydata:`

🔗 Docker Bind Mounts

Your host machine meets your container

👨💻 **DEV TEAM STORY**

A Netflix dev is building a Node.js API. She changes code → rebuilds image → restarts container. That's 45 seconds wasted. Per change. She does 200 changes a day. That's 2.5 hours wasted!

✅ **Bind Mount to the rescue:**

```
docker run -v $(pwd):/app -p 3000:3000 myapp
```

Bind Mount vs Named Volume — Know the Difference

Feature	Bind Mount	Named Volume
Storage location	Host filesystem (you control)	Docker manages (/var/lib/docker)
Portability	❌ Tied to host path	✅ Works anywhere
Use case	Dev: live reload configs	Prod: databases, app data
Performance	Slower on Mac/Windows	Faster, OS-optimized
Security	Full host access ⚠️	Isolated & safer ✅

🎬 Netflix Case Study — When to Use Which?

Real scenarios, real choices

🗄️ Database Team

PostgreSQL storing 50M user watch histories

Named Volume

Data must survive restarts. Docker manages it safely.

```
-v postgres_data:/var/lib/postgresql/data
```

💻 Frontend Dev

Building React UI locally, needs hot-reload

Bind Mount

Code lives on host → container sees changes instantly.

```
-v $(pwd)/src:/app/src
```

🎥 Streaming Engine

Temp buffer for video transcoding (throw away after job)

Anonymous Volume

Scratch space. No persistence needed. Deleted with container.

```
-v /tmp/transcode
```

🔐 Config / Secrets

Injecting nginx.conf and SSL certs from host

Bind Mount (read-only)

Config files live on host, mounted read-only for safety.

```
-v /etc/nginx:/etc/nginx:ro
```

Docker Container Memory

Stop your container from eating your server alive

THE INCIDENT

Netflix ran a recommendation engine container with no memory limits. A memory leak caused it to consume all 64GB of RAM on the server. Every other container — including payments — crashed. 14-minute outage. $2M lost.

--memory (-m)

Hard Limit

Container CANNOT exceed this. If it tries, the OOM killer terminates it.

```
docker run -m 512m myapp
```

Like a strict spending cap on a credit card.

--memory-reservation

Soft Limit

Guaranteed minimum memory. Docker will try to keep at least this much available.

```
docker run --memory-reservation 256m myapp
```

Like a reserved parking spot — yours when you need it.

--memory-swap

Swap Limit

Total memory + swap. Set equal to -m to disable swap. Set -1 for unlimited swap.

```
docker run -m 512m --memory-swap 1g myapp
```

Like overflow parking — slower but prevents crash.

Golden Rule: Always set -m (hard limit) AND --memory-reservation (soft). Example: reservation = 50% of limit.

Quick Recall

Everything We Covered

Docker Prune

Clean dangling images with docker system prune -a. Schedule it nightly.

Named Volume

Use for databases, uploads, anything that must survive restarts.

Volume Types

Named (persist), Anonymous (temp), Bind Mount (dev/config).

Bind Mount

Use for local dev (live reload) and injecting host configs read-only.

Named > Unnamed

Always name your volumes. Anonymous volumes = data loss risk.

Memory Limits

Always set -m hard limit + --memory-reservation soft limit in prod.